



< L'ORIENTÉ OBJET ? />

WORKSHOP C++

Préliminaires

Langage : C++

Compilateur : `clang++` (C++17)

Pas de panique : tout le monde repart de zéro sur le C++.

✓ Compilez vos fichiers avec :

```
g++ -std=c++17 *.cpp -o main  
./main
```

- ✓ Faites attention aux warnings.
- ✓ Lisez attentivement les messages d'erreur.
- ✓ Posez des questions.
- ✓ Cassez des trucs.



Ce workshop est un workshop de **découverte**.
Le but n'est **PAS** de maîtriser le C++ en 2h.

Fil rouge : vous allez construire, bloc par bloc, un mini système de RPG.

Chaque section de cours est immédiatement suivie d'une tâche qui applique ce qui vient d'être expliqué.

Bootstrap — Premier fichier C++



Delivery : hello.cpp



En C, on code avec des **fonctions** et des **structs**.
En C++, on y ajoute des **classes** : des structs qui ont aussi des **comportements** (méthodes).
La nouveauté, c'est la façon d'**organiser** le code.

Créez un fichier `hello.cpp` contenant :

```
#include <iostream>
#include <string>

int main()
{
}
```

Remplissez le main, voici le comportement attendu :

```
Hello , EPITECH!
```

Compilez et exécutez. Observez les différences avec le C :

- `std::string` à la place de `char[]`
- `std::cout` à la place de `printf`
- pas de `#include <stdio.h>` — on inclut `<iostream>`

Concepts :

- `std::string`
- `std::cout`
- namespace `std::`
- compilation C++

1.2 — Pourquoi la Programmation Orientée Objet ?

Imaginons un RPG. En C, les données et le comportement sont séparés :

```
char hero_name[64];
int hero_hp;
int hero_power;

void hero_take_damage(int *hp, int amount)
{
    *hp -= amount;
}
```

En C++, une **classe** regroupe données ET comportements au même endroit :

```
class Hero {
public:
    std::string name;
    int hp;
    int power;

    void takeDamage(int amount)
    {
        hp -= amount;
    }
};
```

C'est le principe d'**encapsulation** : tout ce qui concerne un héros est dans `Hero`.
On crée ensuite des **instances** (objets) de cette classe :

```
Hero arthur;
arthur.name = "Arthur";
arthur.hp = 100;
arthur.takeDamage(20);
```

Task 00 — Hello Hero



Delivery : hero.cpp

Créer votre première classe `Hero` avec :

- un attribut `name` (string) ;
- un attribut `hp` (int) — initialisé à 100 ;
- une méthode `display()` affichant l'état du héros.

Comportement attendu :

```
[Arthur] HP: 100  
[Merlin] HP: 100
```

Concepts :

- `class`
- méthode publique
- instantiation

1.3 — Anatomie d'une classe : private, constructeur, destructeur

Jusqu'ici les attributs de `Hero` sont publics — n'importe qui peut écrire `arthur.hp = -999`. En C++, on protège les données avec `private` et on contrôle l'accès via des méthodes :

```
class Hero {
private: // personne ne peut toucher ça directement
    std::string _name;
    int _hp;

public: // interface visible de l'extérieur
    Hero(const std::string &name); // constructeur
    ~Hero(); // destructeur

    const std::string &getName() const; // getter - le const final
    int getHp() const; // interdit la modification de l'objet
    void takeDamage(int amount);
};
```

Constructeur : appelé automatiquement à la création.

Destructeur : appelé automatiquement à la destruction.

`const` après une méthode : cette méthode ne modifie pas l'objet.

Sur la **pile** (stack), le destructeur est appelé automatiquement à la fin du scope.

Sur le **tas** (heap, avec `new`), il faut appeler `delete` soi-même.

```
{
    Hero arthur("Arthur"); // pile - destructeur appelé à la fin du bloc
}

Hero *merlin = new Hero("Merlin"); // tas
delete merlin; // OBLIGATOIRE - sinon fuite mémoire
```

Task 01 — Encapsulation



Delivery : hero.cpp (modifiez la classe Hero)

Rendez les attributs de `Hero` **privés** et ajoutez :

- `getName()` et `getHp()` (getters `const`) ;
- `takeDamage(int amount)` — les HP ne peuvent pas descendre en dessous de 0 ;
- `heal(int amount)` — les HP ne peuvent pas dépasser 100 ;
- `isAlive()` — retourne `true` si `HP > 0`.

Comportement attendu :

```
[Arthur] HP: 100
[Arthur] HP: 55
[Arthur] HP: 85
Arthur is alive? Yes
[Arthur] HP: 0
Arthur is alive? No
```



Un héros à 0 HP doit rester à 0, pas en négatif.
Un héros à 100 HP healé de 50 reste à 100.

Concepts :

- `private`, `public`
- getters `const`
- protection des invariants

Task 02 — Cycle de vie



Delivery : hero.cpp (modifiez la classe Hero)

Ajoutez un constructeur qui affiche un message à la création et un destructeur qui affiche un message à la destruction.

Créez un héros sur la **pile** et un sur le **tas**, et observez l'ordre d'appel des destructeurs.

Comportement attendu :

```
Arthur sets off for an adventure.  
Merlin sets off for an adventure.  
Merlin is back to his crops.  
Arthur is back to his crops.
```



Sur le tas (**new**), le destructeur n'est appelé que si vous faites **delete**.
Oubliez et vous avez une **fuite mémoire**.

Concepts :

- constructeur / destructeur
- pile (**stack**) vs tas (**heap**)
- **new** / **delete**
- ordre LIFO de destruction sur la pile

1.4 — Héritage et Polymorphisme

Un Warrior et un Mage sont tous les deux des héros, mais ils se battent différemment. L'**héritage** permet de créer des classes spécialisées à partir d'une classe de base.

```
class Hero { // classe de base
public:
    virtual int attack() = 0; // méthode virtuelle pure – DOIT être implémentée
    virtual ~Hero() {} // destructeur virtuel – toujours mettre ça
};

class Warrior : public Hero {
public:
    int attack() override // override signale qu'on redéfinit une virtuelle
    {
        std::cout << "strikes with his sword!" << std::endl;
        return 20;
    }
};
```

= 0 rend la méthode **virtuelle pure** : `Hero` devient **abstraite**, on ne peut plus en créer d'instance directement.

C'est une façon de dire : "tout héros DOIT savoir attaquer, mais chacun à sa façon".

Le **polymorphisme** permet ensuite de manipuler des objets différents via un pointeur sur la classe de base :

```
Hero *party[] = { new Warrior("Arthur"), new Mage("Merlin") };
for (Hero *h : party)
    h->attack(); // C++ choisit la bonne méthode automatiquement
```

Task 03 — Les classes de personnages



Delivery : warrior.cpp, mage.cpp, priest.cpp (+ hero.cpp)

Rendre **Hero** **abstraite** et créer trois classes héritant de **Hero**.

Chaque héros a des **Power Points (PP)**, initialisés au constructeur, bornés entre 0 et 100.

Si un héros manque de PP ou est KO, il affiche "[name] is out of power." ou "[name] is out of combat.".

Warrior :

- `int attack()` — coût : 10 PP — dégâts : 20 — affiche "[name] strikes with his sword."
- `void rest()` — restaure 50 PP — affiche "[name] eats."
- Construction : "[name] vows to protect the kingdom."
- Destruction : "[name] takes off his armor."

Mage :

- `int attack()` — coût : 0 PP — dégâts : 0 — affiche "[name] doesn't know how to fight."
- `int special()` — coût : 50 PP — dégâts : 99 — affiche "[name] casts a fireball."
- `void rest()` — restaure 100 PP — affiche "[name] meditates."
- Construction : "[name] learns magic from his spellbook."
- Destruction : "[name] closes his spellbook."

Priest hérite de **Mage** :

- `void rest()` — restaure 100 PP **et** 100 HP — affiche "[name] prays."
- Construction : "[name] enters the order."
- Destruction : "[name] finds peace."

Comportement attendu :

```
Arthur sets off for an adventure.
Arthur vows to protect the kingdom.
Arthur strikes with his sword.
Arthur is out of power.
Arthur eats.
Arthur strikes with his sword.
Arthur takes 50 damage.
Arthur takes off his armor.
Arthur is back to his crops.
```

Concepts :

- `virtual`, `override`, `= 0`
- classe abstraite
- `protected` pour les attributs partagés entre parent et enfant
- héritage multi-niveaux (`Priest -> Mage -> Hero`)

Task 04 — Interface et Polymorphisme



Delivery : `icharacter.hpp` + tous les fichiers précédents

Créer une interface `ICharacter` avec **uniquement** des méthodes virtuelles pures :

```
class ICharacter {
public:
    virtual ~ICharacter() {}
    virtual const std::string &getName() const = 0;
    virtual int getHp() const = 0;
    virtual int getPower() const = 0;
    virtual int attack() = 0;
    virtual int special() = 0;
    virtual void rest() = 0;
    virtual void takeDamage(int amount) = 0;
};
```

Faire hériter `Hero` de `ICharacter`.

Créer un tableau de pointeurs `ICharacter*` contenant un Warrior, un Mage et un Priest.
Itérer dessus et appeler `attack()` sur chacun.

Comportement attendu :

```
-- Party attacks! --
Arthur strikes with his sword.
Merlin doesn't know how to fight.
Trichelieu doesn't know how to fight.
```



`virtual ~ICharacter(){} est indispensable. Sans destructeur virtuel, delete sur un ICharacter* ne va pas appeler le bon destructeur — vos objets ne seront pas correctement détruits.`

Concepts :

- interface (classe 100% virtuelle pure)
- `virtual ~` (destructeur virtuel)
- dispatch dynamique
- manipulation par pointeur sur classe de base

1.5 — Design Patterns : Singleton et Factory

Un **design pattern** est une solution réutilisable à un problème courant en architecture logicielle. Ce ne sont pas des bibliothèques : ce sont des **recettes** de conception.

Famille	Rôle	Exemples
Créationnel	Comment créer les objets	Singleton, Factory, Builder
Structurel	Comment composer les objets	Adapter, Decorator, Facade
Comportemental	Comment les objets communiquent	Observer, Strategy, Command

Singleton — Un seul exemplaire

Garantit qu'une classe n'a **qu'une seule instance**, accessible globalement.

Cas d'usage : un logger de combat qui tourne pendant toute la partie — un seul suffit, accessible de partout.

```
class GameLogger {
public:
    static GameLogger &getInstance()
    {
        static GameLogger instance; // créé une seule fois
        return instance;
    }
    void log(const std::string &msg) { std::cout << "[LOG] " << msg << std::endl; }

private:
    GameLogger() {} // constructeur privé
    GameLogger(const GameLogger &) = delete; // pas de copie
};

// Utilisation :
GameLogger::getInstance().log("Arthur takes 20 damage.");
```

Factory — Déléguer la création

Centralise la création d'objets dans une seule classe, sans que le reste du code sache quelle classe concrète est instanciée.

Imaginez un jeu qui charge des niveaux depuis un fichier. Il contient "hero": "warrior" ou "hero": "mage". Sans Factory, votre `main` est rempli de `if/else`. Avec Factory :

```
ICharacter *h = HeroFactory::create("warrior", "Arthur", 50);  
// Le main ne sait pas que c'est un Warrior - il voit juste un ICharacter*
```

```
HeroFactory  
- create(type, name, power)  
  -> "warrior" -> new Warrior(name, power)  
  -> "mage"    -> new Mage(name, power)  
  -> "priest"  -> new Priest(name, power)  
  -> ???      -> nullptr
```

Task 05 — Design Pattern : Singleton



Delivery : game_logger.cpp

Implémenter le `GameLogger` en Singleton.

Le `GameLogger` :

- ne peut être instancié qu'**une seule fois** ;
- expose `log(const std::string &message)` affichant `[LOG] message` ;
- expose `getInstance()` (méthode statique) retournant l'instance unique.

Modifier `Hero::takeDamage()` et les méthodes d'attaque pour appeler `GameLogger::getInstance().log(...)` à chaque action de combat.

Comportement attendu :

```
Same instance? Yes
[LOG] Arthur takes 20 damage.
[LOG] Merlin casts a fireball.
[LOG] Arthur is out of combat.
```



Le constructeur doit être **privé**.

`GameLogger(const GameLogger &)= delete;` — on ne copie pas un Singleton.

Concepts :

- Singleton pattern
- méthode statique
- constructeur / copie privés

Task 06 — Design Pattern : Factory



Delivery : hero_factory.cpp

Créer une `HeroFactory` avec une méthode statique :

```
static ICharacter *create(const std::string &type, const std::string &name, int power);
```

Le `main` ne doit **plus jamais** écrire `new Warrior(...)` directement — tout passe par la factory.

Types reconnus : `"warrior"`, `"mage"`, `"priest"`. Pour tout autre type, retourner `nullptr`.

Comportement attendu :

```
[HeroFactory] Creating warrior 'Arthur'...
[HeroFactory] Creating mage 'Merlin'...
[HeroFactory] Creating priest 'Trichelieu'...
[HeroFactory] Unknown type 'necromancer', returning nullptr.
[Arthur] HP: 100 | PP: 50
[Merlin] HP: 100 | PP: 80
[Trichelieu] HP: 100 | PP: 60
```



Si `create()` retourne `nullptr`, vérifiez-le avant d'utiliser le pointeur.
C'est toute la valeur de la Factory : **un seul endroit** gère la logique de création.

Concepts :

- Factory pattern
- méthode statique
- polymorphisme + création découplée
- `nullptr` check

Rappels importants

Structure typique : séparer .hpp et .cpp

```
// Hero.hpp
class Hero : public ICharacter {
public:
    Hero(const std::string &name, int power);
    virtual ~Hero();
    const std::string &getName() const;
    int getHp() const;

protected:
    std::string _name;
    int _hp;
    int _power;
};

// Hero.cpp
#include "Hero.hpp"

Hero::Hero(const std::string &name, int power) : _name(name), _hp(100), _power(power)
{
    std::cout << _name << " sets off for an adventure." << std::endl;
}
```

Mémoire : règle de base

```
ICharacter *h = HeroFactory::create("warrior", "Arthur", 50);
// ... utilisation ...
delete h; // OBLIGATOIRE si créé avec new
h = nullptr; // bonne pratique
```


Bonus

Si vous avez terminé en avance (à part si vous êtes littéralement Claude AI) :

- ✓ **Combat** : `void fight(ICharacter &a, ICharacter &b)` qui alterne les attaques jusqu'à ce que l'un soit à 0 HP. Le `GameLogger` trace chaque coup.
- ✓ **MVC** : un `PartyModel` qui stocke les héros, une `PartyView` qui affiche, un `PartyController` qui recrute via la `HeroFactory`.
La règle d'or : le Model ne connaît pas la View. Ils ne se parlent jamais directement.
- ✓ **Smart pointers** : remplacez vos `new` / `delete` par des pointeurs intelligents.

```
// Avant - vous gérez la mémoire à la main
ICharacter *h = HeroFactory::create("warrior", "Arthur", 50);
delete h;

// Après - destruction automatique à la fin du scope
#include <memory>
std::unique_ptr<ICharacter> h = HeroFactory::create("warrior", "Arthur", 50);
```

- ✓ `std::unique_ptr<T>` : un seul owner, destruction automatique.
- ✓ `std::shared_ptr<T>` : plusieurs owners, destruction quand le dernier disparaît.

Note finale

Le C++ est difficile parce qu'il combine **la puissance du C** (mémoire, performance) avec **la complexité de l'OOP**.

Les design patterns ne sont pas des règles absolues : ce sont des **outils**.

Un bon développeur sait **quand les appliquer** et surtout **quand ne pas les appliquer**.

Comprendre ces concepts aide énormément à comprendre :

- l'architecture des grands projets ;
- pourquoi les frameworks sont conçus comme ils le sont ;
- comment écrire du code **maintenable** et **extensible** ;
- les entretiens techniques en entreprise.

v1

{EPITECH}